

Health Sciences: Final Project

-- Author --

Name: Tanya Samari

Course: CPSC 375: Introduction to Data Science and Big Data

Semester: Spring 2025

This Jupyter Notebook final project explores Ordinary Least Squares (OLS) regression models that can compute the `FFTotal` (Total Fitness Factor Score) based on research data collected from 2002 to 2013 on the physical health of college students aged 18–25. The dataset was provided by the College of Health and Human Development of California State University, Fullerton.

I. Import the Dataset

I loaded the `Health Sciences Data File New (project).csv` dataset and analyzed which variables to consider for calculation of the Total Fitness Factor Score ("`FFTotal`").

```
In [1]: import pandas as pd
import numpy as np
import warnings
import io

from lets_plot import *
from IPython import display

np.set_printoptions(legacy='1.25')
warnings.filterwarnings('ignore')

health_dataset = pd.read_csv('Health Sciences Data File New (project).csv', index_c

# Removing duplicate data, if it exists
health_dataset = health_dataset.reset_index().drop_duplicates(subset='Idnum')

print(health_dataset.columns)

health_dataset.sample(5)
```

```
Index(['Idnum', 'Date', 'Sex', 'Age', 'Ht', 'Wt', 'RF 2', 'RF 3', 'RF 4',
      'RF 5', 'BIA_percent_Fat', 'SF 1', 'SF 2', 'SF 3', 'Waist', 'FF', 'RGM',
      'LGM', 'VC', 'TA', 'PB', 'SBP', 'DBP', 'HR rest', 'Stages', 'PL 1',
      'HR 1', 'RPE 1', 'PL 2', 'HR 2', 'RPE 2', 'PL 3', 'HR 3', 'RPE 3',
      'FFTotal'],
      dtype='str')
```

Out[1]:

	Idnum	Date	Sex	Age	Ht	Wt	RF 2	RF 3	RF 4	RF 5	...	PL 1	HR 1	RPE 1	PL 2	HR 2
3786	F1127	2006-4	F	19	60.5	93.5	1	3	2	0	...	30	157	13	50	180
5499	F2840	2012-2	F	19	63.0	124.4	1	2	1	0	...	50	138	7	75	155
2247	M2248	2012-4	M	20	64.0	148.0	5	3	1	0	...	50	145	10	100	122
5385	F2726	2011-4	F	18	60.0	106.8	1	2	1	0	...	30	121	7	50	131
466	M0467	2004-2	M	18	70.0	175.5	5	3	1	0	...	50	122	9	100	139

5 rows × 35 columns

Observations: Briefly looking at the above dataset, I observed not all student participants had their PL 3 , HR 3 , or RPE 3 values recorded. When I calculated the number of null values out of 6276 entries in the dataset, approximately 6.1% of rows contained a missing PL 3 , HR 3 , and/or RPE 3 value. A majority of missing values were attributed to variables such as BIA_percent_Fat and Waist :

In [2]: `health_dataset.isna().sum()`

```

Out[2]: Idnum          0
        Date           0
        Sex            0
        Age            0
        Ht             0
        Wt             0
        RF 2           0
        RF 3           0
        RF 4           0
        RF 5           0
        BIA_percent_Fat 4680
        SF 1           1596
        SF 2           1596
        SF 3           1596
        Waist          5528
        FF             0
        RGM            0
        LGM            0
        VC             0
        TA             0
        PB             0
        SBP            0
        DBP            0
        HR rest        0
        Stages         0
        PL 1           0
        HR 1           0
        RPE 1          0
        PL 2           0
        HR 2           0
        RPE 2          0
        PL 3           380
        HR 3           380
        RPE 3          393
        FFTotal        0
dtype: int64

```

II. Variable Selection

For the purposes of this assignment, I considered only the `FFTotal` variable as well as five other variables:

```

In [3]: '''
-- Data Dictionary --
* `Sex`:           Sex (Gender)   F = Female , M = Male
* `Height` (Units: in): Height measured with a stadiometer to the closest 0.25 in o
* `Weight` (Units: lb): Weight measured with an electronic scale to the closest 0.1
* `HR rest` (Units: bpm): Seated, resting heart rate, or self-reported or 72
                        if not measured or self-reported
* `HR 2` (Units: bpm): Heart rate at end of stage 2 - determined by Polar HR monit
* `FFTotal`:       Total Fitness Factor Score

'''

```

```
health = (health_dataset[['Age', 'Sex', 'Ht', 'Wt', 'HR rest', 'HR 2', 'FFTotal']])
health.head()
```

```
Out[3]:
```

	Age	Sex	Ht	Wt	HR rest	HR 2	FFTotal
0	19	M	65.0	195.4	72	148	25
1	19	M	68.5	201.4	72	131	32
2	21	M	69.8	133.8	72	144	39
3	18	M	66.0	148.0	72	157	37
4	23	M	67.0	148.7	72	128	42

I also considered the participants' height and weight to calculate their BMI value in relation to participant age, sex, resting heart rate, and their heart rate at the end of stage 2.

```
In [4]: # Set up the LetsPlot-HTML output
LetsPlot.setup_html(offline=True)

# BMI (in US Units) = Weight/Height^2 * 703
health['bmi'] = (health.Wt/(health.Ht ** 2) * 703)

# The data types of the variables
print(health.dtypes)

# View a statistical summary of the variables in the dataset
health.describe()
```

```
Age          int64
Sex          str
Ht           float64
Wt           float64
HR rest      int64
HR 2         int64
FFTotal      int64
bmi          float64
dtype: object
```

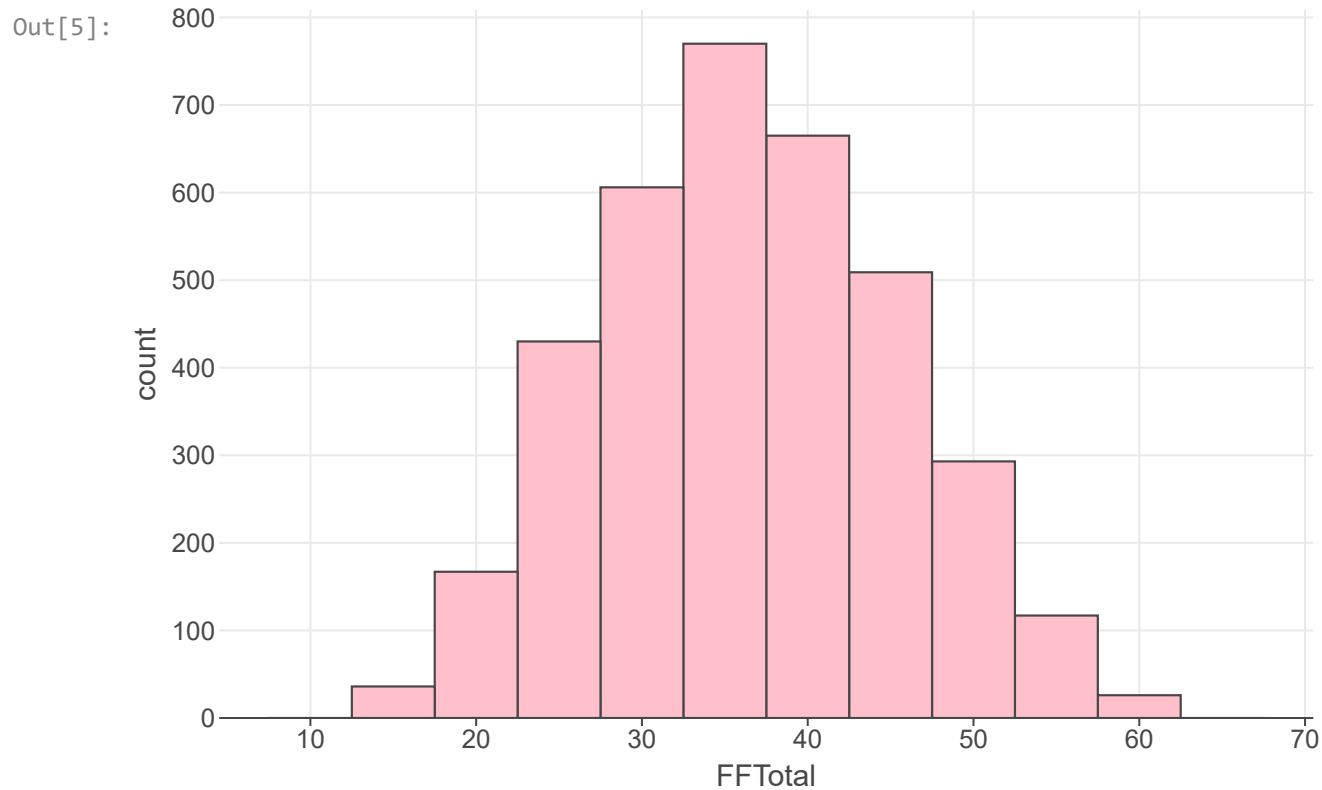
Out[4]:		Age	Ht	Wt	HR rest	HR 2	FFTotal	
	count	6276.000000	6276.000000	6276.000000	6276.000000	6276.000000	6276.000000	6276.
	mean	19.597196	66.210747	147.396303	71.620300	140.423678	37.260357	23.
	std	1.496967	3.820155	33.001523	4.767389	15.742755	9.201547	4.
	min	18.000000	55.000000	82.000000	46.000000	14.000000	11.000000	14.
	25%	19.000000	63.000000	123.300000	72.000000	130.000000	31.000000	20.
	50%	19.000000	66.000000	142.000000	72.000000	139.000000	37.000000	22.
	75%	20.000000	69.000000	166.400000	72.000000	150.000000	44.000000	25.
	max	25.000000	78.000000	329.000000	90.000000	195.000000	62.000000	45.

A. Comparison of the FFTotal Between Female and Male Students

Observations: Based on the two graphs displayed below, I observed 770 of the female students had a fitness factor score of 35 while 560 of the male students had a fitness factor score of 40. However, there was a slight imbalance in the dataset as there were more female participants in the study than male participants.

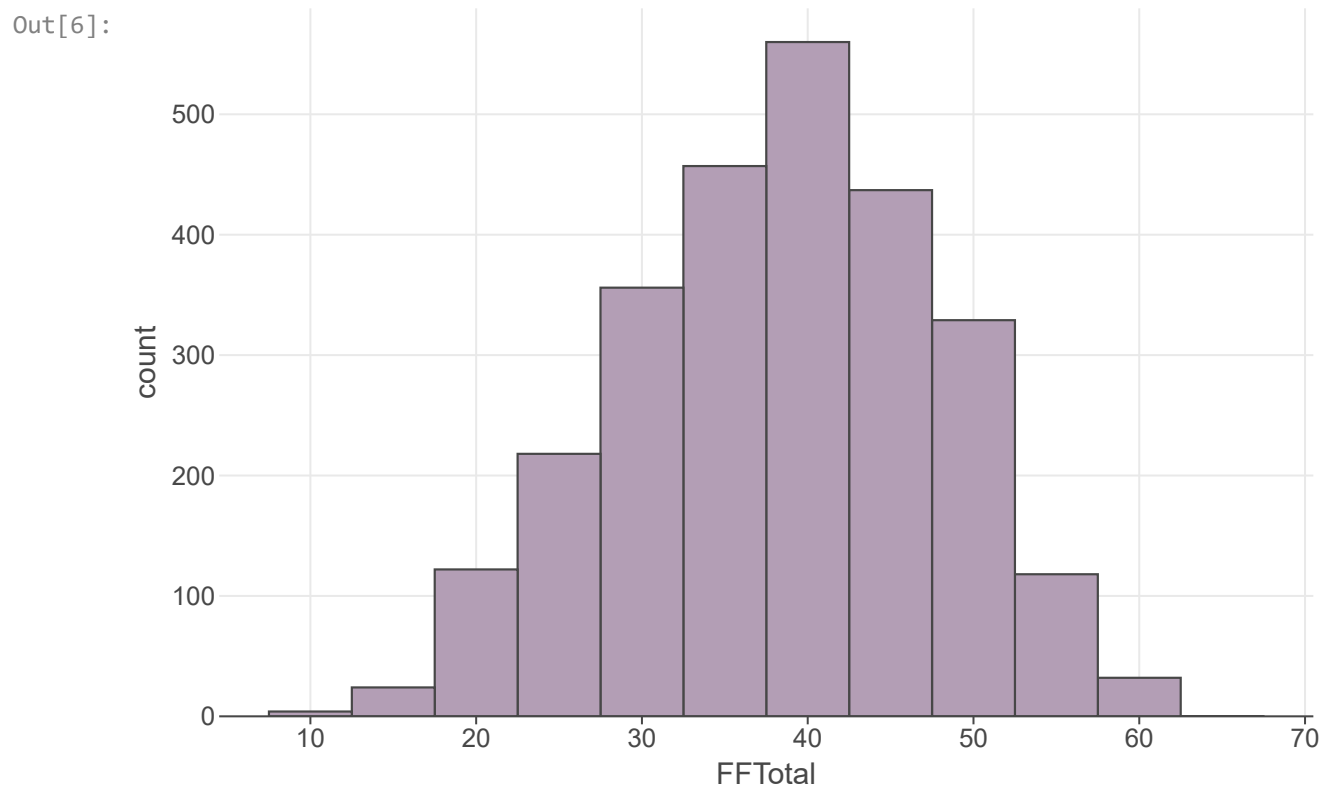
```
In [5]: # Display histogram to show FFTotal distribution of female participants
female_students = health.query('Sex=="F"').copy()

(ggplot(female_students) + geom_histogram(mapping=aes(x='FFTotal'), binwidth=5, cen
```



```
In [6]: # Display histogram to show FFTotal distribution of male participants
male_students = health.query('Sex=="M").copy()

(ggplot(male_students) + geom_histogram(mapping=aes(x='FFTotal'), binwidth=5, fill=
```



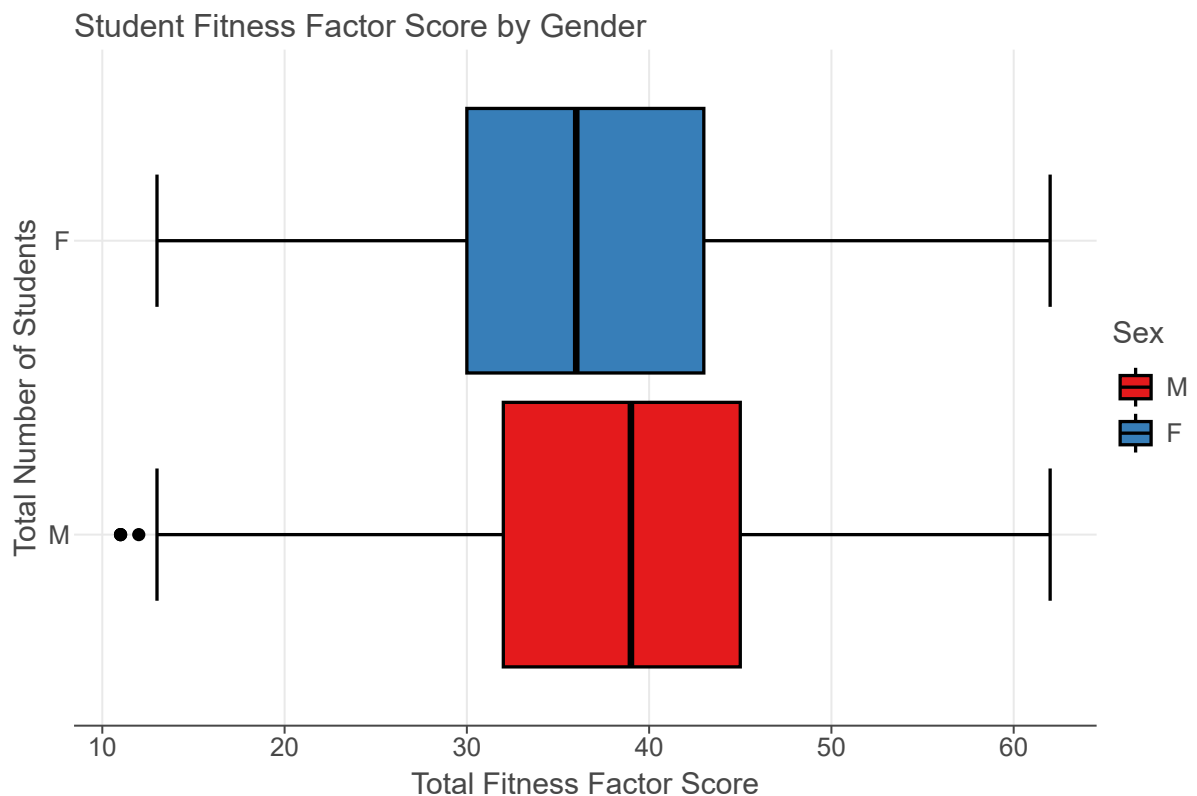
```
In [7]: # Provide an overall summary of the statistics in the dataset
health.describe()
```

Out[7]:

	Age	Ht	Wt	HR rest	HR 2	FFTotal	
count	6276.000000	6276.000000	6276.000000	6276.000000	6276.000000	6276.000000	6276.
mean	19.597196	66.210747	147.396303	71.620300	140.423678	37.260357	23.
std	1.496967	3.820155	33.001523	4.767389	15.742755	9.201547	4.
min	18.000000	55.000000	82.000000	46.000000	14.000000	11.000000	14.
25%	19.000000	63.000000	123.300000	72.000000	130.000000	31.000000	20.
50%	19.000000	66.000000	142.000000	72.000000	139.000000	37.000000	22.
75%	20.000000	69.000000	166.400000	72.000000	150.000000	44.000000	25.
max	25.000000	78.000000	329.000000	90.000000	195.000000	62.000000	45.

```
In [8]: # Display boxplot for student fitness factor score by gender
sex_graph = ( ggplot(data=health) + geom_boxplot(mapping=aes(x='FFTotal', y='Sex',
    labs(title='Student Fitness Factor Score by Gender',
        x='Total Fitness Factor Score',
        y='Total Number of Students') +
    theme(legend_position='right') +
    scale_fill_brewer(palette='Set1') )
sex_graph
```

Out[8]:



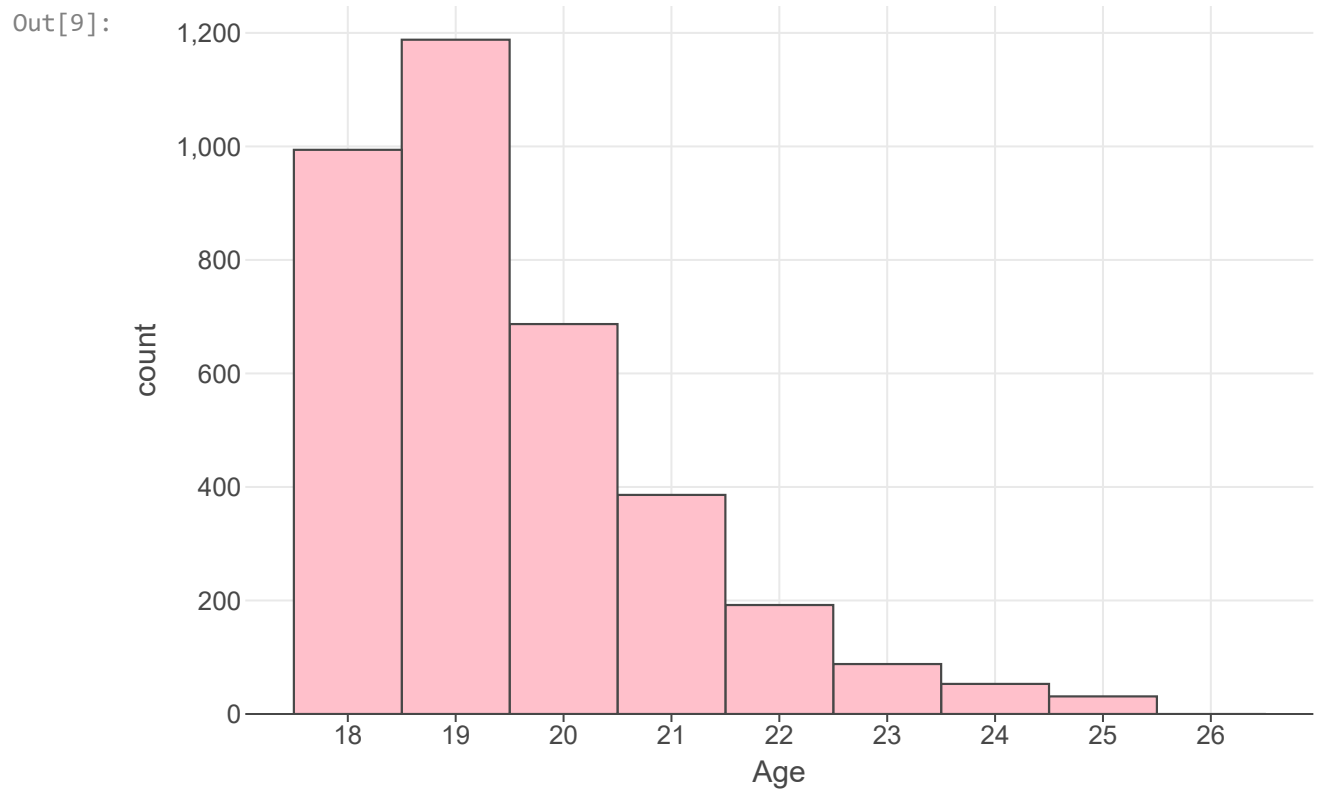
B. Comparison of the Age Distribution Between Female and Male Students

Observations: Based on the two graphs below, a majority of student participants were aged

18–20, with fewer participants aged 21–25. As such, it is possible that the `FFTotal1` may be skewed due to the imbalance in age distribution of the participants.

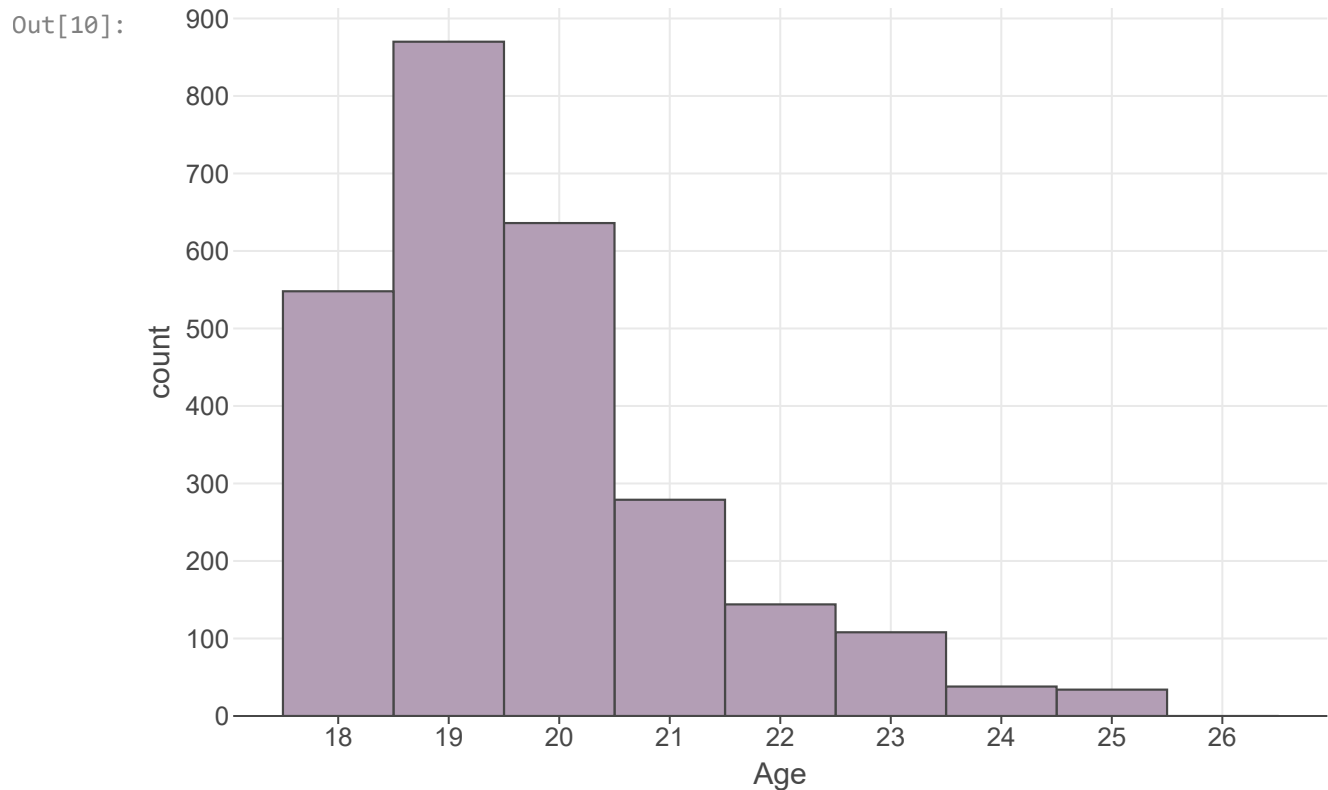
```
In [9]: # Display histogram for age distribution of female students
female_students_age = health.query('Sex=="F"').copy()

(ggplot(female_students_age) + geom_histogram(mapping=aes(x='Age'), binwidth=1, cen
```



```
In [10]: # Display histogram for age distribution of male students
male_students_age = health.query('Sex=="M"').copy()

(ggplot(male_students_age) + geom_histogram(mapping=aes(x='Age'), binwidth=1, cente
```

C. Comparison of HR 2 and HR rest in Relation to FFTotal

Observations: In the graph below, I removed outliers that could potentially skew the results of the research. I found the highest HR 2 rates had the greatest amount of frequencies when the FFTotal ranged between 20 and 40 where the HR 2 results ranged between 108 and 195.

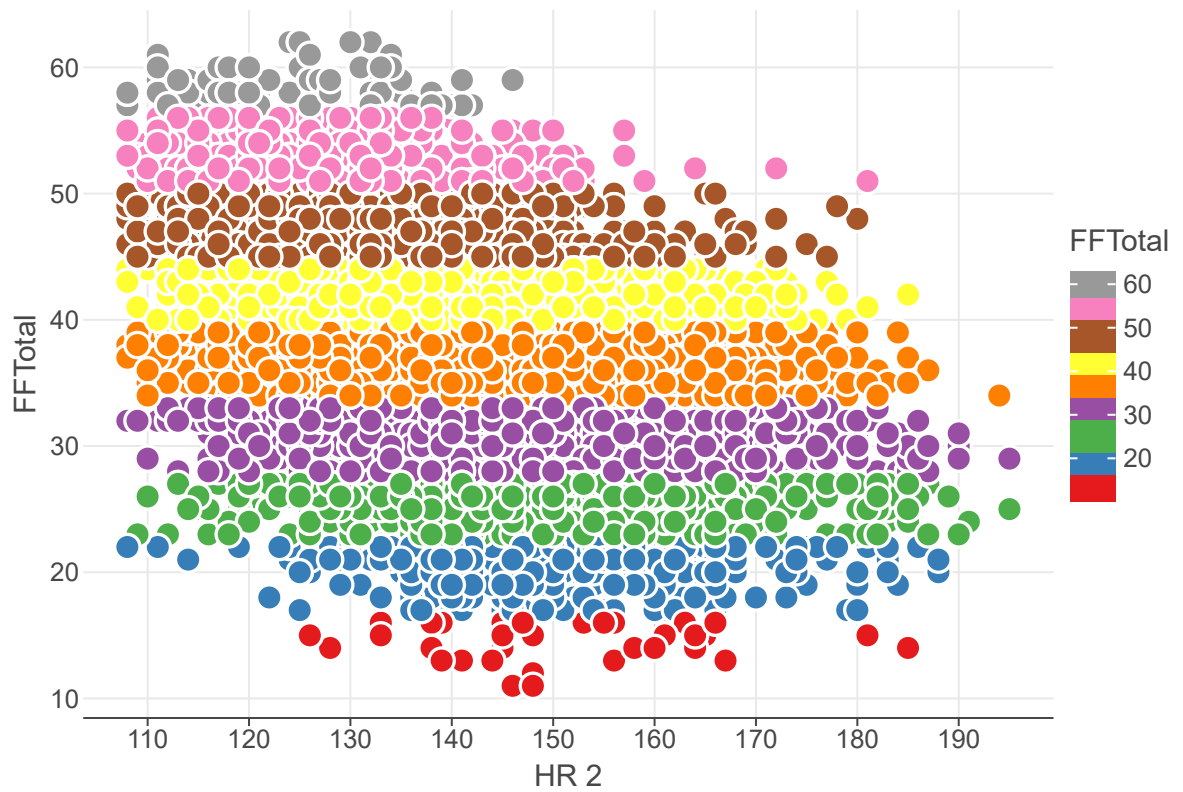
Similarly, I found HR rest had higher rates when the FFTotal was 20 and 49 and the HR rest was between 38 and 90.

```
In [11]: # Filter out outliers in HR 2 and display scatterplot distribution
q = health['HR 2'].quantile(0.01)
health_filtered = health.copy()

hr_2 = health_filtered[health_filtered['HR 2'] > q]
health_filtered['HR 2'] = hr_2['HR 2']

ggplot(data=health_filtered) + geom_point(mapping=aes(x='HR 2', y='FFTotal', fill='
```

Out[11]:

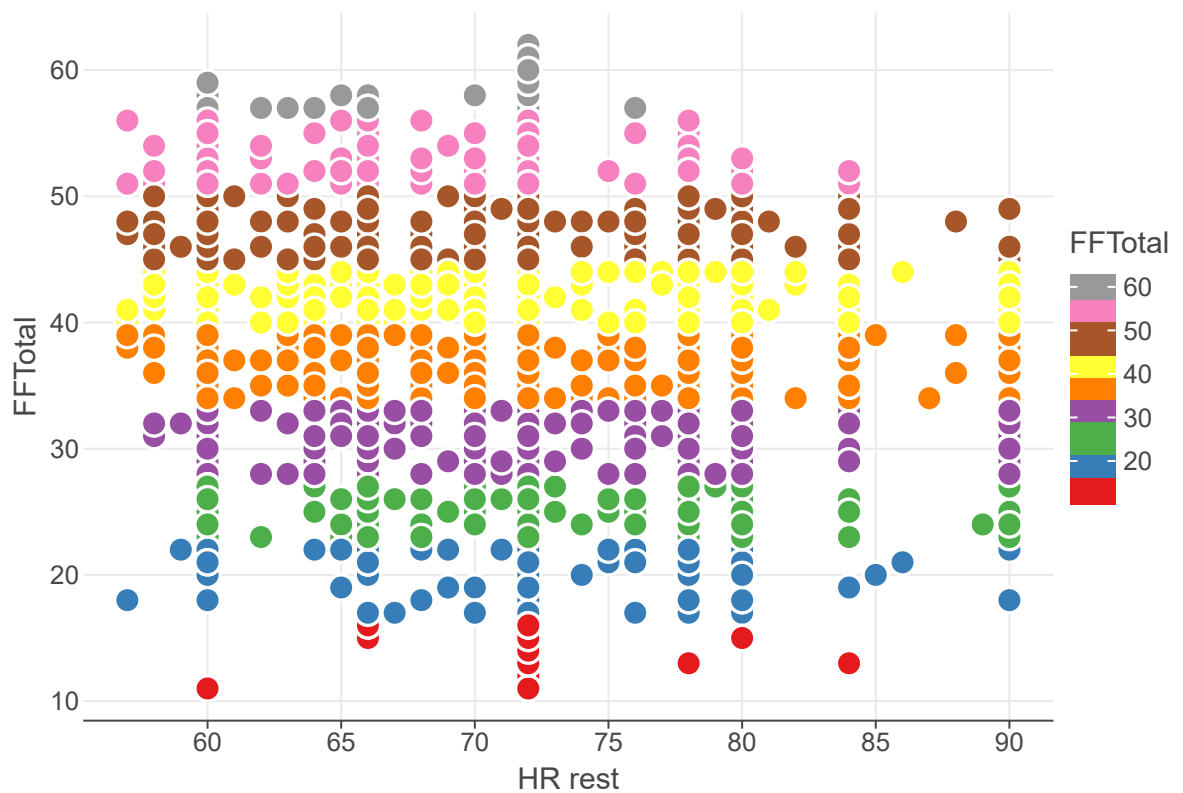


```
In [12]: # Filter out outliers in HR rest and display scatterplot distribution
q_rest = health['HR rest'].quantile(0.01)

hr_rest = health_filtered[health_filtered['HR rest'] > q_rest]
health_filtered['HR rest'] = hr_rest['HR rest']

ggplot(data=health_filtered) + geom_point(mapping=aes(x='HR rest', y='FFTotal', fil
```

Out[12]:



D. Comparison of Student bmi in Relation to FFTotal by Gender

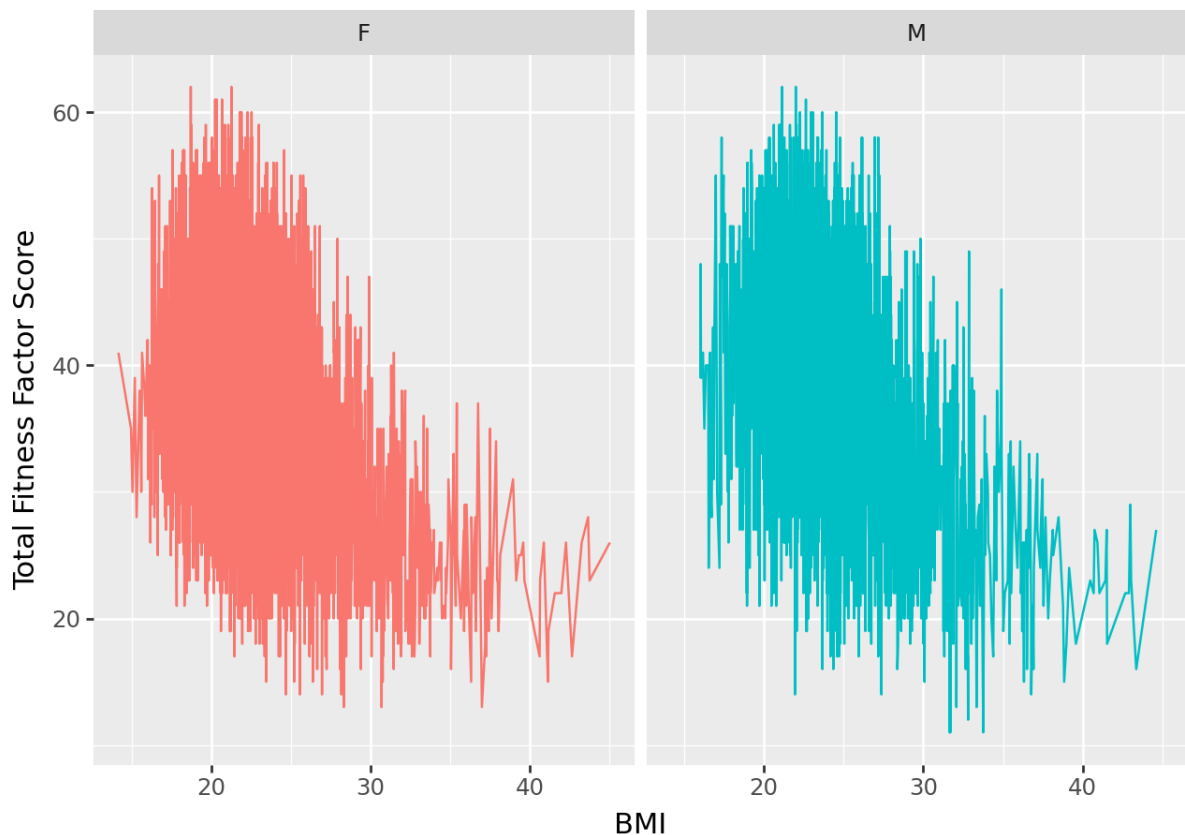
Observations: The line graphs below shows the distribution of BMI in relation to the FFTotal and by gender of the participants. Between these two graphs, I found no significant differences in student BMI based on the participant's gender.

```
In [13]: from plotnine import *

# Display BMI relationship to FFTotal by sex
bmi_graph = (
    ggplot(health, aes('bmi', 'FFTotal', color='Sex'))
    + geom_line(size=.5, show_legend=False)
    + facet_wrap('~Sex')
    + scale_y_continuous()
    + labs(
        x='BMI',
        y='Total Fitness Factor Score',
        title='College Student BMI in Relation to Fitness Factor Score'
    )
)

bmi_graph
```

Out[13]: College Student BMI in Relation to Fitness Factor Score



II. Variable Transformations

After exploring the data, I then selected variables from the dataset into four more variables to be considered during the machine learning process:

- **max_hr:** The maximum total heart rate that is possible for an individual based on their age. This is calculated through the following formula: $208 - 0.7x$, where x represents the individual's age.
- **hrr_lower:** The lower boundary of the target heart rate range. This is first calculated by finding the HRR, or heart rate reserve value ($maxhr - HR_{rest}$), where $(HRR * 0.7) + HR_{rest}$.
- **hrr_upper:** The upper boundary of the target heart rate range. This is first calculated by finding the HRR, or heart rate reserve value ($maxhr - HR_{rest}$), where $(HRR * 0.85) + HR_{rest}$.

```
In [14]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

# Process in which variables are transformed to new variables before
# the machine learning process begins
health_normalized = health_filtered.fillna(0).copy()

# Calculating the maximum total heart rate that is possible based on
# the participant's age
health_normalized['max_hr'] = 206.9 - (health_normalized.Age * 0.7)

# Calculating the heart rate reserve value by the difference between the
# maximum heart rate and the HR rest
hrr = health_normalized['max_hr'] - health_normalized['HR rest']

# Calculating the upper and lower target heart rates based on the participants'
# HR rest values
health_normalized['hrr_lower'] = (hrr * 0.7) + health_normalized['HR rest'] # Minim
health_normalized['hrr_upper'] = (hrr * 0.85) + health_normalized['HR rest'] # Maxi

# Convert Sex to binary values where 'F' = 0 (False) and 'M' = 1 (True)
health_normalized['sex_norm'] = health_normalized.Sex.map({'F': 0, 'M': 1})

health_normalized.sample(10)
```

Out[14]:

	Age	Sex	Ht	Wt	HR rest	HR 2	FFTotal	bmi	max_hr	hrr_lower	hrr_upper
2934	19	F	64.0	114.3	72.0	140.0	38	19.617407	193.6	157.12	175.360
4324	18	F	64.0	117.2	72.0	138.0	47	20.115137	194.3	157.61	175.955
799	19	M	71.0	176.5	60.0	129.0	34	24.614065	193.6	153.52	173.560
3659	18	F	63.0	103.0	60.0	132.0	41	18.243638	194.3	154.01	174.155
389	18	M	71.5	170.4	60.0	143.0	32	23.432187	194.3	154.01	174.155
4407	18	F	65.2	125.8	72.0	127.0	30	20.803709	194.3	157.61	175.955
4244	19	F	64.5	127.4	72.0	170.0	28	21.528081	193.6	157.12	175.360
5025	21	F	66.0	140.2	59.0	130.0	32	22.626400	192.2	152.24	172.220
4269	19	F	64.5	129.6	72.0	129.0	45	21.899838	193.6	157.12	175.360
454	19	M	75.0	189.9	0.0	137.0	42	23.733280	193.6	135.52	164.560

III. Modeling

A. Model #1: Computing the bmi , Wt , Ht , Age , and HR 2 Variables

I used the min-max scaling method to normalize the variables to be used in machine learning, and then I constructed the first OLS model.

```
In [15]: import statsmodels.api as smf
from statsmodels.regression import linear_model

normalizer = MinMaxScaler(copy=False)

X = health_normalized.drop(columns='FFTotal') # Independent variables
y = health_filtered.FFTotal # Dependent variable

X_scaled = X.copy()

# Normalizing independent variables using the min-max scaling method
X_scaled[['bmi']] = normalizer.fit_transform(X_scaled[['bmi']])
X_scaled[['max_hr']] = normalizer.fit_transform(X_scaled[['max_hr']])
X_scaled[['hrr_lower']] = normalizer.fit_transform(X_scaled[['hrr_lower']])
X_scaled[['hrr_upper']] = normalizer.fit_transform(X_scaled[['hrr_upper']])
X_scaled[['HR 2']] = normalizer.fit_transform(X_scaled[['HR 2']])
X_scaled[['Ht']] = normalizer.fit_transform(X_scaled[['Ht']])
X_scaled[['Wt']] = normalizer.fit_transform(X_scaled[['Wt']])
X_scaled[['Age']] = normalizer.fit_transform(X_scaled[['Age']])

# Independent variables to be considered for the first model
X_1 = X_scaled[['bmi', 'Wt', 'Ht', 'Age', 'HR 2']]
```

```

# Splitting the training and testing sets for the first model
(X_train_1, X_test_1, y_train_1, y_test_1) = train_test_split(X_1, y, test_size=0.2)

# Constructing the machine Learning model
X_train_1 = smf.add_constant(X_train_1)
model_1 = linear_model.OLS(y_train_1, X_train_1)
output_1 = model_1.fit()

output_1.summary()

```

Out[15]:

OLS Regression Results						
Dep. Variable:		FFTotal		R-squared:		0.348
Model:		OLS		Adj. R-squared:		0.347
Method:		Least Squares		F-statistic:		534.2
Date:		Mon, 06 Apr 2026		Prob (F-statistic):		0.00
Time:		14:51:31		Log-Likelihood:		-17194.
No. Observations:		5020		AIC:		3.440e+04
Df Residuals:		5014		BIC:		3.444e+04
Df Model:		5				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
const	66.4334	1.734	38.316	0.000	63.034	69.833
bmi	-21.0883	6.618	-3.186	0.001	-34.062	-8.114
Wt	-13.6788	8.398	-1.629	0.103	-30.143	2.785
Ht	10.8703	3.566	3.049	0.002	3.880	17.860
Age	-2.3915	0.490	-4.885	0.000	-3.351	-1.432
HR 2	-33.4677	1.012	-33.072	0.000	-35.452	-31.484
Omnibus:		6.143	Durbin-Watson:		1.983	
Prob(Omnibus):		0.046	Jarque-Bera (JB):		6.026	
Skew:		-0.066	Prob(JB):		0.0491	
Kurtosis:		2.894	Cond. No.		152.	

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [16]: # Compute the RMSE for the first model
np.sqrt(output_1.mse_resid)
```

```
Out[16]: 7.439140740167159
```

Model #1 Evaluation Score

$$\bar{R}^2 \approx 0.347$$

$$RMSE = 7.439140740167159$$

B. Model #2: Computing the bmi , HR rest , Age , hrr_upper , and hrr_lower Variables

```
In [17]: # Independent variables to be considered for the second model
X_2 = X_scaled[['bmi', 'Age', 'HR rest', 'hrr_upper', 'hrr_lower']]

# Splitting the training and testing sets for the second model
(X_train_2, X_test_2, y_train_2, y_test_2) = train_test_split(X_2, y, test_size=0.2)

# Constructing the machine learning model
X_train_2 = smf.add_constant(X_train_2)
model_2 = linear_model.OLS(y_train_2, X_train_2)
output_2 = model_2.fit()

output_2.summary()
```

Out[17]:

OLS Regression Results

Dep. Variable:	FFTotal	R-squared:	0.166
Model:	OLS	Adj. R-squared:	0.166
Method:	Least Squares	F-statistic:	333.5
Date:	Mon, 06 Apr 2026	Prob (F-statistic):	1.73e-197
Time:	14:51:31	Log-Likelihood:	-17809.
No. Observations:	5020	AIC:	3.563e+04
Df Residuals:	5016	BIC:	3.565e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	48.5074	0.909	53.377	0.000	46.726	50.289
bmi	-26.6856	0.876	-30.448	0.000	-28.404	-24.967
Age	1.4419	0.516	2.794	0.005	0.430	2.454
HR rest	-0.2306	0.016	-14.727	0.000	-0.261	-0.200
hrr_upper	11.0950	0.243	45.662	0.000	10.619	11.571
hrr_lower	5.3028	0.116	45.681	0.000	5.075	5.530

Omnibus:	69.618	Durbin-Watson:	1.973
Prob(Omnibus):	0.000	Jarque-Bera (JB):	43.691
Skew:	0.070	Prob(JB):	3.25e-10
Kurtosis:	2.565	Cond. No.	1.31e+17

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 1.5e-27. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
In [18]: # Compute the RMSE for the second model
np.sqrt(output_2.mse_resid)
```

Out[18]: 8.407390634065008

Model #2 Evaluation Score

$$\bar{R}^2 \approx 0.166$$

$$RMSE = 8.407390634065008$$

C. Model #3: Computing the bmi , sex_norm (Sex), Age , max_hr , and hrr_lower Variables

```
In [20]: # Independent variables to be considered for the third model
X_3 = X_scaled[['bmi', 'sex_norm', 'max_hr', 'hrr_upper', 'Age']]

# Splitting the training and testing sets for the third model
(X_train_3, X_test_3, y_train_3, y_test_3) = train_test_split(X_3, y, test_size=0.2)

# Constructing the machine learning model
X_train_3 = smf.add_constant(X_train_3)
model_3 = linear_model.OLS(y_train_3, X_train_3)
output_3 = model_3.fit()

output_3.summary()
```

Out[20]:

OLS Regression Results

Dep. Variable:	FFTotal	R-squared:	0.200			
Model:	OLS	Adj. R-squared:	0.199			
Method:	Least Squares	F-statistic:	250.1			
Date:	Mon, 06 Apr 2026	Prob (F-statistic):	2.84e-239			
Time:	14:51:31	Log-Likelihood:	-17707.			
No. Observations:	5020	AIC:	3.543e+04			
Df Residuals:	5014	BIC:	3.546e+04			
Df Model:	5					
Covariance Type:	nonrobust					
	coef	std err	t	P> t 	[0.025	0.975]
const	-1.751e+13	2.69e+13	-0.651	0.515	-7.02e+13	3.52e+13
bmi	-28.7091	0.870	-32.987	0.000	-30.415	-27.003
sex_norm	3.4683	0.240	14.477	0.000	2.999	3.938
max_hr	1.751e+13	2.69e+13	0.651	0.515	-3.52e+13	7.02e+13
hrr_upper	-8.2722	1.506	-5.493	0.000	-11.224	-5.320
Age	1.751e+13	2.69e+13	0.651	0.515	-3.52e+13	7.02e+13
Omnibus:	42.919	Durbin-Watson:	1.983			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	29.532			
Skew:	0.052	Prob(JB):	3.87e-07			
Kurtosis:	2.639	Cond. No.	6.41e+14			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 3.13e-26. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
In [21]: # Compute the RMSE for the third model
np.sqrt(output_3.mse_resid)
```

Out[21]: 8.23923653528071

Model #3 Evaluation Score

$$\bar{R}^2 \approx 0.199$$

$$RMSE = 8.23923653528071$$

D. Model #4: Computing the bmi , hrr_upper , hrr_lower , max_hr , and HR 2 Variables

```
In [22]: # Independent variables to be considered for the fourth model
X_4 = X_scaled[['bmi', 'hrr_upper', 'hrr_lower', 'max_hr', 'HR 2']]

# Splitting the training and testing sets for the fourth model
(X_train_4, X_test_4, y_train_4, y_test_4) = train_test_split(X_4, y, test_size=0.2)

# Constructing the machine learning model
X_train_4 = smf.add_constant(X_train_4)
model_4 = linear_model.OLS(y_train_4, X_train_4)
output_4 = model_4.fit()

output_4.summary()
```

Out[22]:

OLS Regression Results

Dep. Variable:	FFTotal	R-squared:	0.340
Model:	OLS	Adj. R-squared:	0.340
Method:	Least Squares	F-statistic:	646.0
Date:	Mon, 06 Apr 2026	Prob (F-statistic):	0.00
Time:	14:51:31	Log-Likelihood:	-17223.
No. Observations:	5020	AIC:	3.446e+04
Df Residuals:	5015	BIC:	3.449e+04
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	72.1296	1.123	64.245	0.000	69.929	74.331
bmi	-31.3279	0.790	-39.642	0.000	-32.877	-29.779
hrr_upper	-1.0422	0.571	-1.826	0.068	-2.161	0.077
hrr_lower	-1.6517	0.696	-2.373	0.018	-3.016	-0.287
max_hr	2.7432	0.531	5.166	0.000	1.702	3.784
HR 2	-35.5319	0.978	-36.334	0.000	-37.449	-33.615

Omnibus:	6.436	Durbin-Watson:	1.971
Prob(Omnibus):	0.040	Jarque-Bera (JB):	6.466
Skew:	-0.081	Prob(JB):	0.0394
Kurtosis:	2.932	Cond. No.	1.57e+15

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 7.05e-27. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
In [23]: # Compute the RMSE for the fourth model
np.sqrt(output_4.mse_resid)
```

Out[23]: 7.481022248663421

Model #4 Evaluation Score

$$\bar{R}^2 \approx 0.340$$

$$RMSE = 7.481022248663421$$

E. Model #5: Computing the bmi , Ht , max_hr , hrr_upper , and HR 2 Variables

```
In [24]: # Independent variables to be considered for the fifth model
X_5 = X_scaled[['bmi', 'Ht', 'max_hr', 'hrr_upper', 'HR 2']]

# Splitting the training and testing sets for the fifth model
(X_train_5, X_test_5, y_train_5, y_test_5) = train_test_split(X_5, y, test_size=0.2)

# Constructing the machine learning model
X_train_5 = smf.add_constant(X_train_5)
model_5 = linear_model.OLS(y_train_5, X_train_5)
output_5 = model_5.fit()

output_5.summary()
```

Out[24]:

OLS Regression Results

Dep. Variable:	FFTotal	R-squared:	0.348
Model:	OLS	Adj. R-squared:	0.347
Method:	Least Squares	F-statistic:	534.5
Date:	Mon, 06 Apr 2026	Prob (F-statistic):	0.00
Time:	14:51:31	Log-Likelihood:	-17193.
No. Observations:	5020	AIC:	3.440e+04
Df Residuals:	5014	BIC:	3.444e+04
Df Model:	5		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	67.8245	1.250	54.271	0.000	65.374	70.275
bmi	-31.7328	0.788	-40.294	0.000	-33.277	-30.189
Ht	5.1266	0.669	7.662	0.000	3.815	6.438
max_hr	3.0223	0.586	5.157	0.000	1.873	4.171
hrr_upper	-2.6422	1.370	-1.929	0.054	-5.327	0.043
HR 2	-33.1871	1.019	-32.557	0.000	-35.186	-31.189

Omnibus:	6.316	Durbin-Watson:	1.979
Prob(Omnibus):	0.043	Jarque-Bera (JB):	6.224
Skew:	-0.070	Prob(JB):	0.0445
Kurtosis:	2.898	Cond. No.	28.5

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [25]: # Compute the RMSE for the fifth model
np.sqrt(output_5.mse_resid)
```

Out[25]: 7.4383480420419135

Model #5 Evaluation Score

$$\bar{R}^2 \approx 0.347$$

$$RMSE = 7.4383480420419135$$

IV. Conclusion

Model #	R2 (Adjusted)	RMSE
1	0.347	7.439140740167158
2	0.166	8.40739063406501
3	0.199	8.238120857933527
4	0.340	7.481022248663421
5	0.347	7.4383480420419135

The modeling results suggest the best models for calculating the `FFTotal` were the following two models:

- **Model #1:** The first model in which the independent variables were `bmi` , `Wt` , `Ht` , `Age` , and `HR 2` .
- **Model #2:** The second model in which the independent variables were `bmi` , `Ht` , `max_hr` , `hrr_upper` , and `HR 2` .

While both models had similar R2 values, the fifth model had a slight advantage over the first model due to its slightly lower RMSE value. The fifth model having a slightly lower RMSE value suggests it made less significant mistakes with slightly more accurate predictions.

From what can be observed in this project, an individual's Total Fitness Factor Score can be greatly impacted by **their BMI score** as well as their **age** and **maximum possible heart rate**. Sex did not appear to have significant positive impact on the Total Fitness Factor Score, and when I attempted to introduce sex as a factor in the third model, it resulted in a lower R2 value for the model as well as an increased RMSE value, which suggests the model made more significant mistakes.